ASDXDDL8M2 Fundamentals of SQL Day 1 Summary

Wednesday, October 14th, 2026

Overview

This session advances from earlier design and basic querying topics into grouping, aggregation and result ordering within T-SQL using Microsoft SQL Server and SQL Server Management Studio (SSMS). QA Trainer Ninad Rampure leads the session and active participants include Victoria Warnock-Dundas, David Matthews, Emily Wason, Ewelina Rabeda, Melinda Frohlich, Julia Matheson and others.

Major topics and outcomes

- GROUP BY and aggregation: Ninad explains common aggregate functions (COUNT, SUM, AVG, MIN, MAX) and demonstrates grouping rows by an entity (e.g., CategoryID) to compute per-group summaries such as average unit price; he shows ordering those grouped results and filtering groups using HAVING (noting HAVING cannot use aliases).

- Query clause order and interaction: Ninad reiterates the execution order and placement of clauses—SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY—emphasizing WHERE filters rows before grouping and HAVING filters after grouping; he uses examples (average price per category and HAVING AVG(UnitPrice) > 50) to show practical usage.

- Practical labs and troubleshooting: Learners complete timed exercises on filtering, sorting, limiting and aggregation; Ninad assists with syntax/debugging, reminds GoToMyPC/LOD considerations for accessing files, and advises saving queries in Documents to avoid issues.

Commitments and follow-ups

- QA Trainer Ninad Rampure offers to review individual learners' problematic queries and to join their machines for debugging (applied for Ewelina Rabeda earlier in the session).

- Ninad will cover functions and more advanced JOINs in the next session beginning tomorrow morning and will check learners' progress at the scheduled checkpoint.

Concise impact

Attendees consolidate practical T-SQL skills for aggregating and grouping data, learn how HAVING differs from WHERE, and prepare to apply scalar and aggregate functions plus JOINs in upcoming sessions to support multi-table analysis and reporting.

Detailed Summary

Excel vs database

Trainer contrasts using Excel for business data with using a relational database. He frames a scenario of a small English-teaching company that started with an Excel file for employee details and explains why Excel becomes problematic as the business grows. The discussion introduces the need to manage structure, privacy, and scalability as data complexity increases.

- QA Trainer Ninad Rampure explains that structured data in Excel (rows and columns) can be filtered but asks why a database might be better for growing businesses.

- QA Trainer Ninad Rampure uses a small company example to show Excel works initially but can create problems as requirements expand, such as tracking car registrations and promotions.
- Ewelina Rabeda highlights a confidentiality concern when colleagues can see colleagues' job titles and salaries in a shared spreadsheet.
- Emily Wason demonstrates a simple Excel filter by car registration as a basic example of a query.

Data privacy and confidentiality

The trainer raises data protection concerns that emerge when a shared spreadsheet exposes unnecessary employee details. He points out potential HR and equality issues when salaries and sensitive fields are visible to all. The group acknowledges that access control and limiting visible attributes are required as a solution.

- QA Trainer Ninad Rampure identifies that unrestricted access to employee rows exposes salary and role details and notes implications for data protection.
- Ewelina Rabeda explicitly calls this a confidentiality problem when Walter can view all employee details.
- QA Trainer Ninad Rampure suggests role-based access control as a mechanism to allow limited queries (e.g., name and contact only) without exposing full HR data.

Data redundancy and anomalies

The trainer demonstrates how representing multiple related facts (e.g., one employee with multiple cars) in a single spreadsheet forces row duplication and creates update anomalies. He explains how duplicated records lead to inconsistent or outdated information and unreliable queries. The group recognizes the need to avoid redundancy for data integrity.

- QA Trainer Ninad Rampure shows a scenario where James has two car registrations and must be duplicated in Excel, creating redundancy.
- Julia Matheson and others observe duplicated entries cause data to be not up to date and inaccurate, as noted by QA Trainer Ninad Rampure.
- QA Trainer Ninad Rampure labels this an 'update anomaly' that creates inconsistency when a single attribute (e.g., salary) must be updated in multiple rows.

Introducing relational design

Participants propose using unique identifiers and splitting data into related tables. The trainer explains primary keys, foreign keys, and normalization: separate employee, car, and salary tables link by IDs to avoid duplication and control access. The conversation clarifies when to create separate tables based on entities and attributes relevant to the business domain.

- Chris Mclellan suggests using a unique ID to distinguish people, which QA Trainer Ninad Rampure endorses.
- David Matthews recommends splitting into two tables (employee and car) so a person appears once and can be linked to multiple cars, as reiterated by QA Trainer Ninad Rampure.

- QA Trainer Ninad Rampure defines primary keys (unique identifiers) for each table and foreign keys to reference related records from other tables.

Deciding table boundaries

The trainer and participants discuss how far to normalize data and when to create separate tables for attributes like car make or job role. Ninad explains entities versus attributes and ties the design decision to the business context and how many attributes describe an entity. He signals deeper data modeling sessions will cover relationship mapping and normalization trade-offs.

- Crawford asks when normalization should stop and whether make/color should be separate tables, prompting discussion.
- QA Trainer Ninad Rampure explains that tables reflect business entities and that attributes (like make or color) may remain attributes unless the business treats them as entities needing separate descriptions.
- QA Trainer Ninad Rampure commits to teaching data modeling and management later to cover entity identification and relationship design.

Relational databases benefits

The trainer summarizes why relational databases make multi-entity queries, access control, and joins simpler than managing multiple Excel files. He stresses the benefits for combining data from customers, appointments, products, and treatments without complex cross-file formulas. The group accepts relational DBs as the better solution for growing, connected datasets.

- QA Trainer Ninad Rampure highlights that relational databases let you map entities (e.g., patient, doctor, appointment) and join tables with primary keys to perform multi-table queries easily.
- QA Trainer Ninad Rampure notes that relational databases provide table-level access control to limit who can query sensitive entities, unlike a shared Excel file.
- QA Trainer Ninad Rampure emphasizes that a single query can pull related data from different tables, unlike juggling multiple Excel workbooks with complex formulas.

Labs access setup

Before beginning hands-on work, Ninad walks learners through accessing lab environments (GoToMyPC and LOD), login procedures, and common connection problems. He collects names of blocked users, instructs on creating a launcher shortcut, and explains differences between LOD and GoToMyPC behavior. He also describes how redeem keys will be delivered for LOD users.

- QA Trainer Ninad Rampure instructs learners to check direct chat for login details and to copy username/password and access codes exactly without extra spaces.
- QA Trainer Ninad Rampure asks blocked participants to report so he can request licenses, and he notes several users (e.g., Melinda, Julia, Mary) are blocked by their organizations.

- QA Trainer Ninad Rampure tells GoToMyPC users to create a desktop shortcut via File > Create Shortcut to avoid repeated lockouts, and he commits to requesting training keys for LOD users during the break.

Troubleshooting connection issues

Participants report various connectivity and launcher problems; the trainer advises waiting for lockouts to expire, using correct PC selection (student number), and alternative approaches like launching via AWS for one learner. The trainer keeps a running list of affected learners to follow up and ensures most participants access the lab desktop before continuing.

- David Matthews reports successful connection via an AWS browser instance after local launcher issues, which QA Trainer Ninad Rampure acknowledges as an acceptable workaround.
- QA Trainer Ninad Rampure explains that repeated failed logins can lock accounts for five minutes and asks participants to avoid repeated attempts during lockouts.
- Marie asks about LOD training key timing and QA Trainer Ninad Rampure confirms he will post the key in chat during the break.

LOD vs GoToMyPC differences

Ninad explains platform-specific behaviors: LOD desktops run in a browser and adapt to browser size, restrict direct copy-paste (workaround via virtual keyboard), and have a reset schedule; GoToMyPC supports direct copy-paste and needs server name selection for the SQL connection. He provides practical tips to maximize desktop area and use the instruction panel.

- QA Trainer Ninad Rampure tells LOD users to maximize the browser window and remove the instruction panel to increase desktop size.
- QA Trainer Ninad Rampure explains LOD restricts direct clipboard copy-paste and instructs users to use the platform's virtual keyboard 'type text' feature to paste content.
- QA Trainer Ninad Rampure reminds GoToMyPC users that their server name differs (student PC name) and instructs them to use 'Browse more' > Data Engine to locate the correct SQL server name.

Locating course files and software

The trainer instructs learners to open the lab PC file explorer, find the 'course files' folder and the lab workbook (Fundamentals to SQL), and launch SQL Server Management Studio. He clarifies which server name to use for LOD (MIS) and classroom (student PC name) and emphasizes maximizing SSMS so the taskbar and course files remain accessible.

- QA Trainer Ninad Rampure directs learners to C:\ course files \ lab book \ Fundamentals to SQL to access workbooks, solutions, and slides.
- QA Trainer Ninad Rampure instructs all learners to launch SQL Server Management Studio and to trust the service certificate when prompted.
- QA Trainer Ninad Rampure tells GoToMyPC users to select their PC's data engine name in the SSMS server dropdown before connecting and to maximize SSMS so the bottom taskbar remains visible.

SQL server introduction

Ninad defines Microsoft SQL Server and places it among other DBMS vendors, explaining that SQL is the query language used to retrieve and manipulate structured data; he introduces SQL Server Management Studio (SSMS) as the tool to connect to and manage SQL Server instances. He clarifies that SSMS offers both graphical and code-based management but that data manipulation is performed via queries.

- QA Trainer Ninad Rampure identifies Microsoft SQL Server as the relational database used in the course and names alternative vendors such as Oracle, MySQL, PostgreSQL, MariaDB, MongoDB, and Cosmos DB.
- QA Trainer Ninad Rampure explains that SQL is a standardized query language (ANSI/ISO) and that Transact-SQL (T-SQL) is Microsoft's dialect for SQL Server.
- QA Trainer Ninad Rampure describes SQL Server Management Studio as a centralized tool that can connect to multiple SQL Servers, provide an Object Explorer, and support both graphical configuration and script-based management.

Query context and SSMS workflow

Ninad explains how queries operate inside SSMS: you must select the database context (e.g., USE Northwind) so queries target the correct database, and you execute queries (Execute or F5) to see results. He also explains object discovery through SSMS suggestions and the importance of saving query files for labs.

- QA Trainer Ninad Rampure instructs learners to specify the active database (e.g., USE Northwind) to ensure queries run against the intended dataset.
- QA Trainer Ninad Rampure demonstrates that SSMS suggests table and column names when a FROM clause is present and encourages saving query windows per lab instructions.
- QA Trainer Ninad Rampure highlights that execution feedback appears at the bottom of SSMS, showing row counts and completion messages after running a query.

SELECT and FROM basics

The trainer teaches the fundamental SELECT ... FROM pattern for retrieving data: SELECT specifies which columns to return and FROM specifies the source table. He covers SELECT * for all columns, selecting specific columns with commas, and making use of SSMS autocomplete to pick table and column names efficiently.

- QA Trainer Ninad Rampure demonstrates SELECT * FROM Products to return all 77 product rows and explains that SELECT * returns every column from the chosen table.
- QA Trainer Ninad Rampure advises listing specific columns separated by commas when you only want particular attributes, and he shows using keyboard arrows to choose suggested column names.
- QA Trainer Ninad Rampure notes that column names with special characters or SQL reserved words require square brackets to be treated as identifiers.

Expressions and calculated columns

Ninad shows how to create expressions in SELECT to compute values (arithmetic and string operations) and how to assign aliases using AS to name derived columns. He demonstrates arithmetic addition for stock totals and string concatenation to form full names, and emphasizes using brackets and aliases for clarity.

- QA Trainer Ninad Rampure uses arithmetic expressions (e.g., UnitsInStock + UnitsOnOrder) to produce a computed 'FutureStock' column and names it with AS.
- QA Trainer Ninad Rampure demonstrates string concatenation for employee first and last name (with explicit space characters) to produce a 'FullName' column and warns that omitting the space will run names together.
- QA Trainer Ninad Rampure recommends single-word aliases and square brackets when aliases or identifiers include spaces or special characters.

Coding style and comments

The trainer recommends consistent formatting and commenting practices to make SQL scripts readable and maintainable. He covers case insensitivity of SQL keywords, single-line and block comment syntax, and four-part naming conventions for tables when querying across databases or servers.

- QA Trainer Ninad Rampure notes that SQL is not case sensitive but recommends consistent capitalization for readability and debugging.
- QA Trainer Ninad Rampure explains comment syntax using double hyphens for single-line comments and /* ... */ for block comments to document intent in scripts.
- QA Trainer Ninad Rampure discusses naming conventions (two-, three-, and four-part names) and suggests using two-part names (schema.table) like dbo.Employee for most local queries and three- or four-part names when joining across databases or servers.

Retrieving data labs and exercises

Ninad outlines the lab practicals where learners save queries and complete three tasks: retrieve all columns, select specific columns, and calculate stock value using expressions. He sets expectations for verifying outcomes (e.g., 77 rows for Products) and points learners to solutions if they get stuck.

- QA Trainer Ninad Rampure instructs learners to open module two in the workbook, start new query tabs, save them with specified names, and run tasks in sequence.
- QA Trainer Ninad Rampure describes task verification: task one should return 77 product rows, task two should present specified columns, and task three requires an arithmetic expression to compute stock value.
- QA Trainer Ninad Rampure sets a timed lab interval and encourages learners who finish early to try optional exercises such as additional concatenation tasks, and he points to the solution folder for guidance.

Filtering rows (WHERE clause)

The trainer introduces filtering rows using the WHERE clause to restrict returned rows based on conditions. He explains comparison, logical and special operators (e.g., =, , BETWEEN, IN, IS NULL) and demonstrates filtering out discontinued products by testing a Discontinued flag.

- QA Trainer Ninad Rampure defines WHERE as the clause that applies row-level conditions after specifying SELECT and FROM.
- QA Trainer Ninad Rampure demonstrates filtering with WHERE Discontinued = 0 to return only active products (69 rows) and contrasts that with running the same query without the WHERE clause (77 rows).
- QA Trainer Ninad Rampure explains comparison operators (>, =, vs >=).

Advanced WHERE usage

Trainer expands WHERE coverage with logical grouping, special operators (IN, BETWEEN, LIKE), NOT usage, and handling NULLs. He emphasizes correct operator precedence, use of parentheses to control evaluation order, and practical patterns for date ranges and wildcard string matches. The session closes with lab guidance to practice these filters across several exercises.

- Victoria Warnock-Dundas explains she queries date ranges across many Azure-fed databases and uses BETWEEN to target records between two dates.
- Emily Wason asks about filtering rows within a 90-day window and QA Trainer Ninad Rampure confirms using date comparisons or BETWEEN to express a relative date range.
- QA Trainer Ninad Rampure demonstrates combining conditions with AND/OR and shows that parentheses control evaluation order, attributing the analogy to David Matthews' point about operator precedence.
- QA Trainer Ninad Rampure shows IN as a compact alternative to multiple ORs and illustrates BETWEEN for numeric ranges, noting both produce equivalent outcomes when framed correctly.
- QA Trainer Ninad Rampure explains LIKE with wildcards (%) to match patterns (starts with, contains, ends with) and confirms it's case-insensitive for these examples.
- QA Trainer Ninad Rampure cautions against overusing NOT because it can be confusing and explains NOT IN as a way to exclude specific values, as suggested by David Matthews.
- QA Trainer Ninad Rampure explains NULL semantics and shows IS NULL is the correct way to find missing values, highlighting scenarios (demographics, missing contact details) where identifying NULLs matters for data quality.
- QA Trainer Ninad Rampure notes WHERE cannot reference column aliases because the alias is computed after WHERE is evaluated, and he demonstrates replacing aliases with the full expression when filtering.

Labs: WHERE practice and troubleshooting

Learners work through lab exercises focused on equality, IN/BETWEEN, LIKE, and NULL filters; trainer times the lab, offers solution guidance, and troubleshoots date-format and syntax issues. He encourages using solutions when queries become complex and offers to check learners' queries individually after the session.

- QA Trainer Ninad Rampure instructs learners to copy column names from lab instructions to save typing and to save queries under specified names.
- QA Trainer Ninad Rampure warns US date format and surrounding quotes can cause date-filter failures and offers to review learners' queries with typos or bracket errors, as noted when Ewelina Rabeda reports trouble.
- QA Trainer Ninad Rampure schedules a short break and sets expectations to resume at five past three to complete exercises three and four and optional tasks for early finishers.
- QA Trainer Ninad Rampure advises learners to consult provided solutions for complicated multi-clause queries to learn structuring patterns and avoid losing confidence.
- QA Trainer Ninad Rampure offers to join individual learners' machines to debug syntax issues, as demonstrated when he corrected Ewelina Rabeda's unclosed bracket.

Sorting rows (ORDER BY)

Ninad introduces ORDER BY to arrange filtered results and explains it must come after SELECT, WHERE, and any grouping; default order is ascending and DESC specifies descending. He covers ordering by single and multiple columns, ordering by expressions and aliases, and notes ORDER BY should be the last clause to minimize unnecessary system work on large datasets.

- QA Trainer Ninad Rampure states ORDER BY is always the final clause and that ascending is the default unless DESC is specified.
- QA Trainer Ninad Rampure demonstrates single-column ordering (e.g., unit price DESC) to list most expensive items first and explains multi-column ordering groups by the first column then sorts the next within each group.
- QA Trainer Ninad Rampure confirms ORDER BY can use computed expressions or aliases (unlike WHERE) so you can sort by calculated columns like FutureStock.
- QA Trainer Ninad Rampure explains ordering by up to sixteen columns and uses category ID then unit price as an example to show most expensive items per category.

Distinct and limiting results

Trainer shows DISTINCT to remove duplicate rows from query results and introduces TOP as a way to return a specific number of rows, noting TOP may exclude tied values and requires careful use for ranking scenarios. He contrasts DISTINCT behaviour with non-distinct results and explains combining DISTINCT with ORDER BY for aggregated listings like unique cities per country.

- QA Trainer Ninad Rampure demonstrates DISTINCT to collapse repeating country/city rows into unique entries and shows the row count reduction that results.

- QA Trainer Ninad Rampure notes TOP returns the first N rows after ordering and warns it can omit tied values with the same ranking, so users should design queries accordingly.

TOP and WITH TIES

The trainer explains TOP behaviour in ranking scenarios and introduces WITH TIES to include tied rows beyond the numeric limit. He shows practical examples using product prices and discusses when ties are appropriate to avoid unfair exclusion of equally ranked items.

- QA Trainer Ninad Rampure explains that TOP N returns exactly N rows and that identical scores or values can cause seemingly arbitrary exclusion of tied items when ties are not considered.
- QA Trainer Ninad Rampure demonstrates TOP with WITH TIES returns additional rows that have the same sort key value as the last included row and shows an example where TOP 11 WITH TIES returned 12 products because two shared the same price.
- QA Trainer Ninad Rampure recommends using WITH TIES when fairness matters for ranking (e.g., top products by popularity) and using plain TOP when a strict count limit is required.

GROUP BY and aggregation

Ninad introduces aggregate functions (COUNT, SUM, AVG, MIN, MAX) and explains how GROUP BY collects rows into groups so aggregates operate per group. He clarifies that GROUP BY belongs after WHERE, that HAVING filters groups (not individual rows), and that ORDER BY is applied last; practical examples show average unit price per category and filtering groups with HAVING.

- QA Trainer Ninad Rampure explains aggregate functions like AVG, COUNT, MIN and MAX are used in SELECT to compute summary values across rows.
- QA Trainer Ninad Rampure demonstrates GROUP BY category to compute the average unit price per category and then uses ORDER BY to rank categories by that average, attributing the ordering to the final step.
- QA Trainer Ninad Rampure states HAVING applies conditions to grouped results and attributes an example where HAVING AVG(UnitPrice) > 50 returns only categories meeting that threshold.
- QA Trainer Ninad Rampure notes HAVING cannot use column aliases and that HAVING always follows GROUP BY while ORDER BY comes last.

Labs progress and next steps

Trainer checks lab completion status, gives guidance for opening saved queries from the Documents folder, and decides to cover GROUP BY briefly because many learners finished early. He sets expectations to resume with JOINs and functions in the next session and offers to review outstanding lab work during a quick checkpoint.

- QA Trainer Ninad Rampure instructs learners to open previously saved query files from the Documents\SQL Studio folder to avoid opening errors via SSMS.

- QA Trainer Ninad Rampure confirms several learners have completed the exercises and offers to teach a short GROUP BY overview now to leave more time for joins and functions later.
- QA Trainer Ninad Rampure schedules a three-twenty check-in to assess progress and commits to covering JOINs and functions starting tomorrow morning.